



Electrical and Computer Engineering

ENCS5325 - Wireless Sensor Networks and Internet of Things

**Smart-Home IoT Network Design and
Performance Evaluation Using OMNeT++**

Student: Nooreen Abuhmoud

Student ID: 1222470

Semester: Summer 2026

1. Project Overview

The project builds and evaluates a smart-home IoT network using OMNeT++ and the INET Framework. The constrained access side uses the IEEE 802.15.4 narrowband model. Smart-home devices communicate through an IoT gateway to an IP/core-side monitoring application. The model extends the official INET IEEE 802.15.4 smart-home showcase and is evaluated using controlled parameter sweeps and multi-seed statistics.

The project specification requires at least two traffic classes: periodic monitoring traffic and event-driven/critical traffic. The baseline implementation verifies both classes end-to-end. Experiment 1 isolates network-size effects using periodic monitoring traffic; Experiment 3 separately evaluates the urgent smoke-alarm traffic under increasing background load.

2. Simulation Environment and Implemented Model

Item	Current implementation
Simulator	OMNeT++ 6.3.0
Framework	INET 4.6.0
Access technology	IEEE 802.15.4 narrowband model in INET
Application/transport/network stack	UDP over IPv4 on the simulated end-to-end path
Smart-home side	Parameterized sensor nodes plus actuator nodes and one IoT gateway
Core side	Monitoring server connected to the gateway through a PPP/IP link
Periodic traffic	50-byte application payload every 10 s to UDP port 1000
Critical traffic	20-byte smoke alarm from sensor[4] at $t = 50$ s to UDP port 2000
Application-strategy experiment	Request/response polling: monitoring server UdpBasicApp clients to sensor UdpEchoApp servers; publish/subscribe-like: sensor UdpBasicApp publishers to a monitoring-server UdpSink subscriber.

Implementation note: The implemented end-to-end stack uses IPv4/UDP over IEEE 802.15.4. The cross-layer study therefore separates two evidence types: (1) simulated packet-size effects in the current INET model, and (2) an analytical 6LoWPAN compression/fragmentation estimate based on RFC 4944 and

RFC 6282. FRAG1/FRAGN counts are not claimed as simulator outputs; they are reported explicitly as analytical approximations.

3. Experimental Methodology

Key comparisons use five independent repetitions with seed-set = $\{\text{repetition}\}$. Experiments 1 and 3 use 100 s per run; Experiments 2, 4, and 5 use 600 s per run to provide sufficient samples at low reporting rates and across the payload-size sweep. Reported summaries are computed from OMNeT++ scalar/statistical results and exported with opp_scavetool.

For Experiment 3, the ordinary background workload remains fixed at a 50-byte report every 10 s from each active sensor, while the number of active sensors is swept over $N = 5, 10, 20, 30,$ and 40 . Sensor[4] additionally generates one 20-byte CriticalSmokeAlarm at $t = 50$ s. A project-defined end-to-end critical-event latency requirement of less than 100 ms is tested against all completed repetitions.

For Experiment 4, the number of active sensors is fixed at $N = 12$ and the application payload at 50 B. The interval T is swept over 1, 5, 10, 30, and 60 s for two application strategies. Request/response polling uses one UdpBasicApp client per sensor at the monitoring server and UdpEchoApp on the sensors; its application-visible latency is the round-trip packet lifetime (RTT). The publish/subscribe-like proxy uses UdpBasicApp publishers on the sensors and a UdpSink subscriber at the monitoring server; its latency is one-way end-to-end delay. Application packet count is used as a traffic-cost proxy, and the distinction between RTT and one-way delay is preserved in all comparisons.

4. Experiment 1 - Network Scalability

4.1 Objective

The objective is to determine how increasing the number of active periodic-reporting sensor nodes affects reliability, latency, throughput, packet loss, and energy cost, and to identify whether congestion/contention becomes significant within the required sweep range.

4.2 Controlled Parameters

Parameter	Setting
Active sensor nodes N	5, 10, 20, 30, 40
Application payload	50 B (fixed)
Reporting interval	10 s (fixed)
Simulation duration	100 s per run
Independent repetitions	5 seeds per N

Measured workload	Periodic monitoring traffic only
Destination	Core monitoring server through the IoT gateway

4.3 Metrics and Processing

- Packet Delivery Ratio (PDR) = received packets / sent packets x 100%.
- Packet loss (%) = (sent packets - received packets) / sent packets x 100%.
- End-to-end delay: mean application packet age at the monitoring server, reported in milliseconds.
- Average application throughput = delivered application payload bits / simulation duration, reported in bps.
- Energy per delivered packet = total energy consumed by active sensors plus the IoT gateway / successfully delivered packets. Using the INET showcase convention with `IdealEpEnergyStorage`, consumed energy is obtained as `-residualEnergyCapacity`.

For each N, the table reports the mean across five independent seeds. Standard deviation (SD) is shown for delay and energy/packet; PDR, loss, and average throughput had zero variation across the five runs in the completed sweep.

4.4 Results

Table 1. Experiment 1 results averaged over five independent seeds.

N	PDR (%)	Loss (%)	Delay (ms) Mean +/- SD	Throughput (bps)	Energy/packet (mJ) Mean +/- SD
5	100.0	0.0	5.365 +/- 0.031	200	1.143 +/- 0.001
10	100.0	0.0	5.373 +/- 0.073	400	1.970 +/- 0.000
20	100.0	0.0	5.474 +/- 0.116	800	3.632 +/- 0.000
30	100.0	0.0	5.467 +/- 0.103	1200	5.295 +/- 0.000
40	100.0	0.0	5.565 +/- 0.089	1600	6.961 +/- 0.007

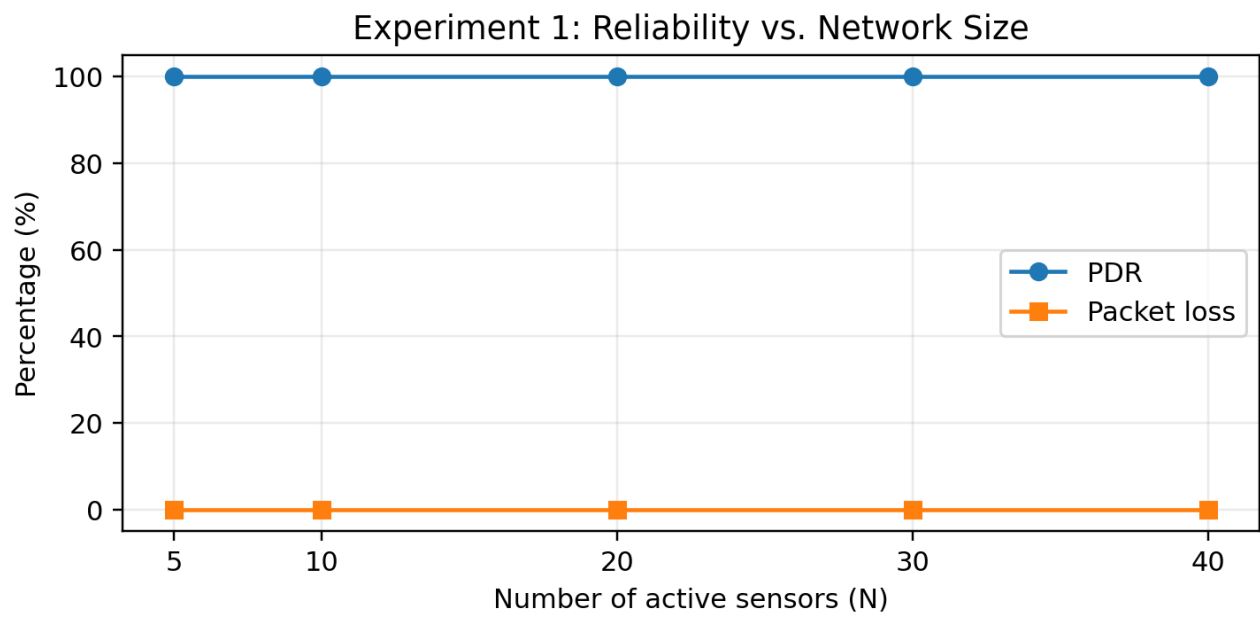


Figure 1. PDR and packet loss versus number of active sensors.

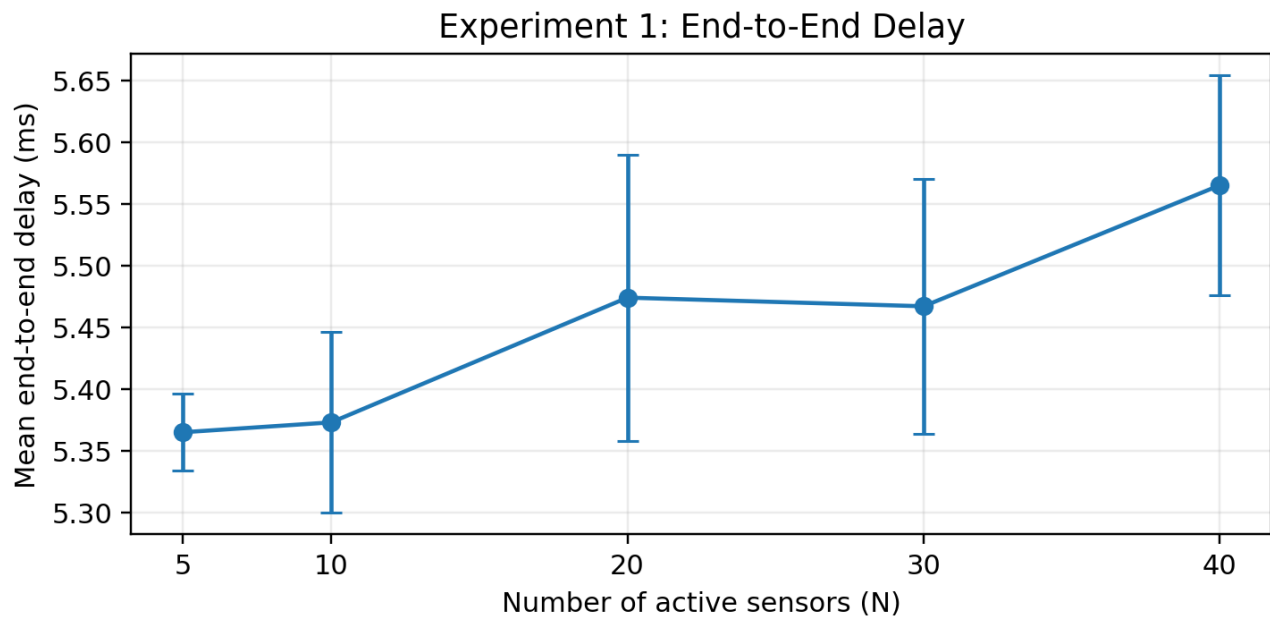


Figure 2. Mean end-to-end delay with one-standard-deviation error bars.

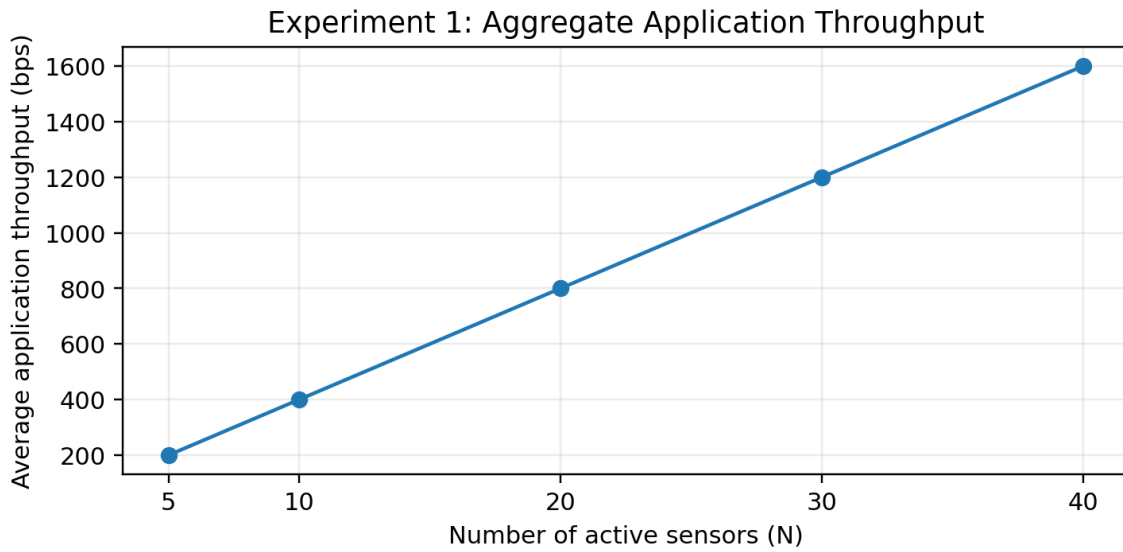


Figure 3. Aggregate application throughput versus number of active sensors.

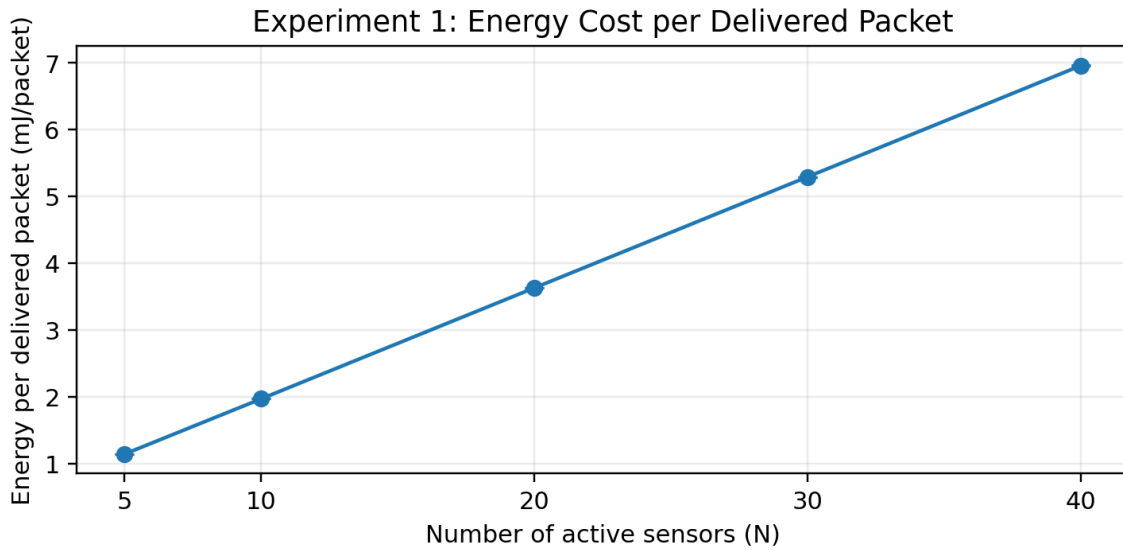


Figure 4. Energy per delivered packet with one-standard-deviation error bars.

4.5 Discussion

Reliability remained perfect throughout the tested scalability range. PDR was 100% and packet loss was 0% for $N = 5, 10, 20, 30$, and 40. Therefore, the tested baseline load did not drive the network into a loss-dominated congestion region.

Mean end-to-end delay showed only a small overall increase as the network grew, from 5.365 ms at $N = 5$ to 5.565 ms at $N = 40$. The small non-monotonic difference between $N = 20$ and $N = 30$ is consistent with random CSMA/CA backoff behavior across seeds. Aggregate application throughput increased linearly from 200 bps to 1600 bps, while energy per delivered packet rose from approximately 1.143 to 6.961 mJ/packet as the active network became denser.

Experiment 1 conclusion: significant congestion/contention was not observed up to $N = 40$ under the tested baseline load of 50-byte periodic reports every 10 s. The network preserved 100% PDR and low delay, while throughput scaled linearly and energy cost increased with the number of active nodes.

4.6 Reproducibility Artifacts

The experiment produced 25 scalar/vector result sets (five N values x five repetitions), plus `experiment1_runs.csv` and `experiment1_summary.csv` for processed per-run and aggregate statistics.

5. Experiment 2 - Reporting Interval and Energy/Latency Trade-off

5.1 Objective

This experiment evaluates how the periodic reporting interval affects delivery reliability, end-to-end latency, and energy cost. The number of active sensors and application payload are fixed while reporting interval T is swept across 1, 5, 10, 30, and 60 s, directly addressing the trade-off between data freshness and network/battery cost.

5.2 Controlled Parameters

Parameter	Setting
Active sensor nodes	12 (fixed)
Application payload	50 B (fixed)
Reporting interval T	1, 5, 10, 30, 60 s
Simulation duration	600 s per run
Independent seeds	5 per reporting interval
Measured workload	Periodic monitoring only; critical-event traffic is evaluated separately in Experiment 3
Access technology	IEEE 802.15.4 narrowband model in INET

5.3 Metrics and Processing

- PDR (%) and packet loss (%), calculated from total application packets sent and received.
- Mean end-to-end delay at the core monitoring server, in milliseconds.

- Total energy consumed by the active sensors plus the IoT gateway, using the INET IdealEpEnergyStorage convention in which consumed energy is the negative of residualEnergyCapacity.
- Energy per successfully delivered packet, in mJ/packet, used as an additional efficiency indicator.
- Average delivered application throughput, retained as a supporting metric to verify the offered-load trend.

5.4 Results

T (s)	PDR (%) Mean +/- SD	Loss (%) Mean +/- SD	Delay (ms) Mean +/- SD	Total energy (J) Mean +/- SD	Energy/packet (mJ) Mean +/- SD
1	99.997 +/- 0.006	0.0028 +/- 0.0063	5.642 +/- 0.236	16.1473 +/- 0.0557	2.260 +/- 0.009
5	100.000 +/- 0.000	0.0000 +/- 0.0000	5.406 +/- 0.015	3.2637 +/- 0.0038	2.275 +/- 0.000
10	100.000 +/- 0.000	0.0000 +/- 0.0000	5.405 +/- 0.022	1.6576 +/- 0.0000	2.302 +/- 0.000
30	100.000 +/- 0.000	0.0000 +/- 0.0000	5.399 +/- 0.044	0.5772 +/- 0.0000	2.405 +/- 0.000
60	100.000 +/- 0.000	0.0000 +/- 0.0000	5.373 +/- 0.045	0.3055 +/- 0.0000	2.546 +/- 0.000

Table 2. Experiment 2 results averaged over five independent seeds.

Supporting mean application throughput decreased with increasing T, as expected from the lower reporting rate: 4763.9, 956.3, 480.0, 160.0, and 80.0 bps for T = 1, 5, 10, 30, and 60 s, respectively.

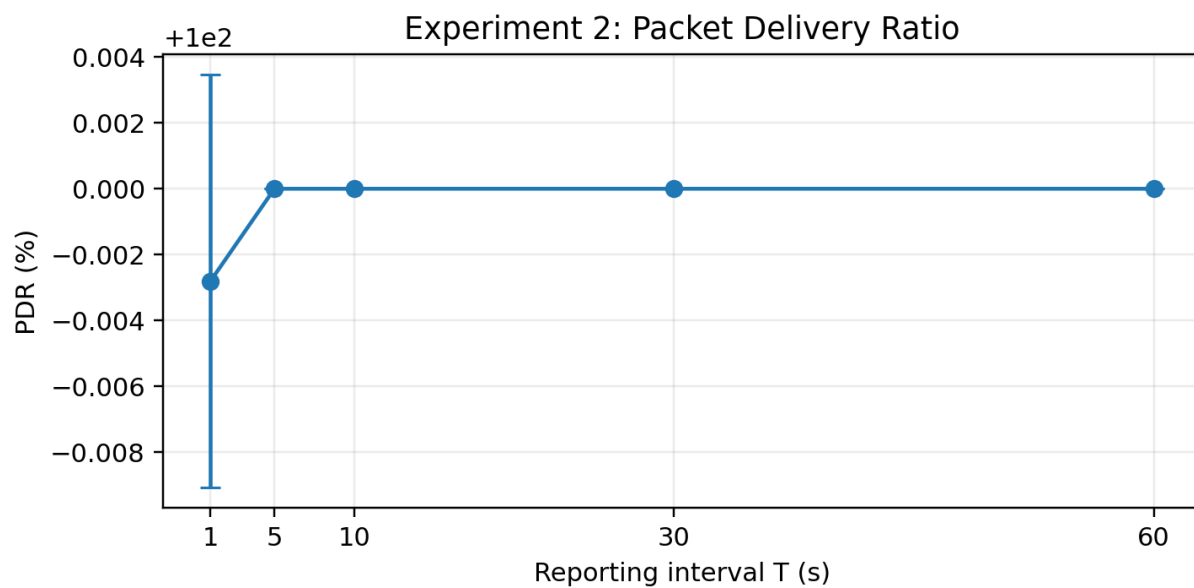


Figure 5. Packet delivery ratio versus reporting interval, with one-standard-deviation error bars.

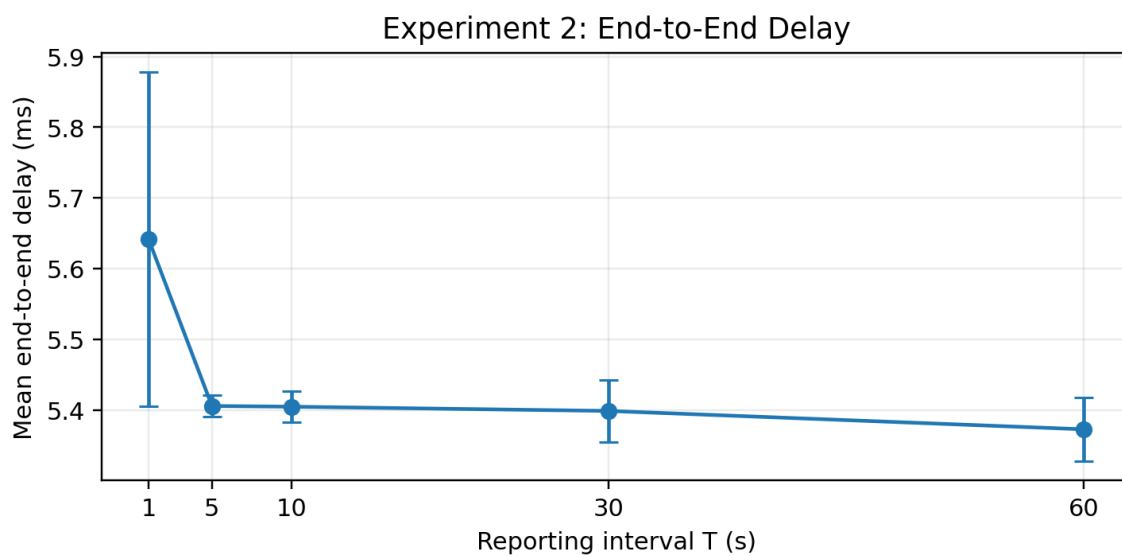


Figure 6. Mean end-to-end delay versus reporting interval, with one-standard-deviation error bars.

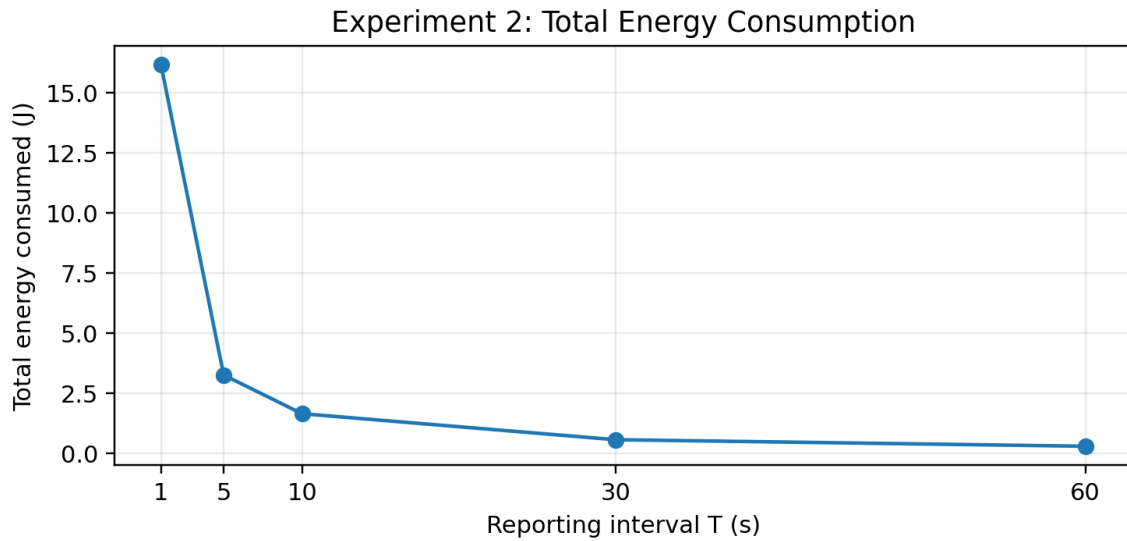


Figure 7. Total energy consumed by active sensors plus the IoT gateway versus reporting interval.

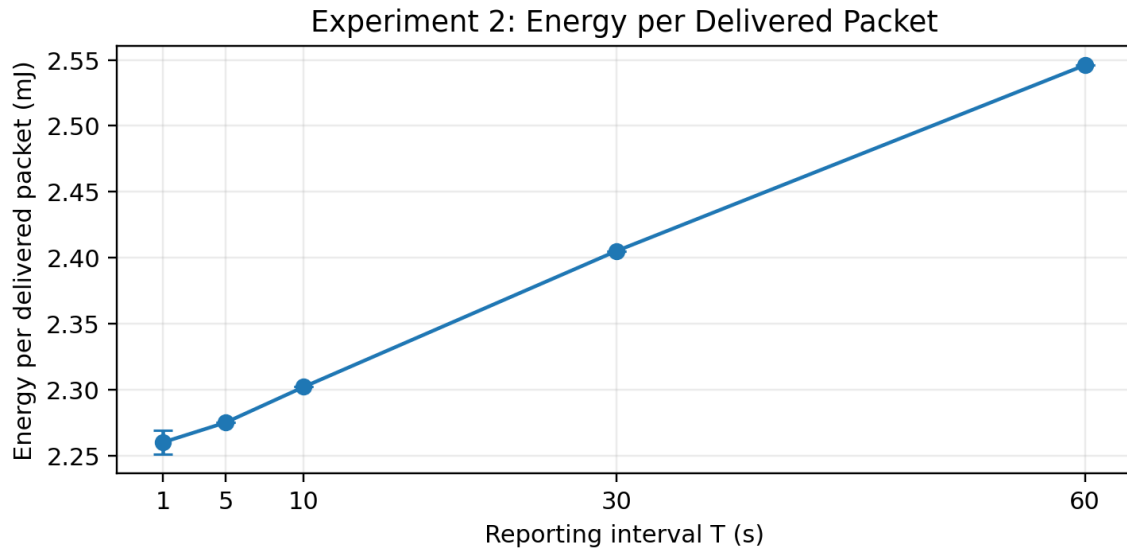


Figure 8. Energy per delivered packet versus reporting interval, with one-standard-deviation error bars.

5.5 Discussion

Reliability remained essentially perfect across the sweep. At $T = 1$ s, the mean PDR was 99.997% and mean loss was only 0.0028%; for $T = 5, 10, 30$, and 60 s, PDR was 100% with zero packet loss. Mean end-to-end delay was also highest at $T = 1$ s (5.642 ms), while for $T \geq 5$ s it stayed close to 5.37-5.41 ms.

The dominant trade-off appears in total energy consumption. Increasing T from 1 s to 60 s reduced mean total energy from 16.1473 J to 0.3055 J, approximately a 98.1% reduction. Energy per delivered packet increased gradually from 2.260 to 2.546 mJ/packet because background/idle radio activity is amortized over fewer delivered packets at longer intervals.

Experiment 2 conclusion: shorter reporting intervals improve data freshness but sharply increase traffic and total energy cost. Longer intervals substantially reduce total energy while retaining 100% delivery in this topology, at the expense of less frequent updates. Mid-range intervals such as 10-30 s provide a practical balance under the tested conditions.

5.6 Reproducibility Artifacts

The experiment produced 25 scalar/vector result sets (five reporting intervals x five repetitions). Processed data are stored in experiment2_runs.csv and experiment2_summary.csv.

6. Experiment 3 - Critical Event under Background Traffic

6.1 Objective

This experiment evaluates urgent-event delivery while ordinary smart-home sensors generate periodic background traffic. The required background load is varied over five levels, and the critical smoke alarm is evaluated for delivery reliability and end-to-end latency. The experiment also tests a project-defined critical-event latency requirement of less than 100 ms.

6.2 Controlled Parameters

Parameter	Setting
Background-load levels	N = 5, 10, 20, 30, 40 active sensors
Background traffic	Each active sensor sends 50 B every 10 s to UDP port 1000
Critical source	sensor[4]
Critical event	One 20 B CriticalSmokeAlarm at t = 50 s to UDP port 2000
Simulation duration	100 s per run
Independent repetitions	5 per N; seed-set = \${repetition}
Critical receiver	monitoringServer.app[1] (UdpSink)
Latency requirement	< 100 ms end-to-end (project-defined engineering target)

6.3 Metrics and Processing

- Critical delivery rate (%) = critical packets received / critical packets sent x 100%; critical loss is the complementary percentage.
- Critical-event end-to-end delay at monitoringServer.app[1], converted from seconds to milliseconds.

- Mean delay and standard deviation across the five independent repetitions at each background-load level.
- Minimum and maximum observed critical-event delay across repetitions. Packet-to-packet jitter is not reported because each run contains only one critical alarm; maximum delay is used instead, as permitted by the project requirement when jitter is not available.

6.4 Results

Table 3. Experiment 3 critical-event results across five independent repetitions per background-load level.

N	Critical delivery (%)	Critical loss (%)	Delay (ms) Mean +/- SD	Min delay (ms)	Max delay (ms)
5	100.0	0.0	3.916 +/- 0.351	3.340	4.300
10	100.0	0.0	4.620 +/- 0.506	3.980	5.260
20	100.0	0.0	4.108 +/- 0.771	3.340	4.940
30	100.0	0.0	4.620 +/- 0.816	3.660	5.580
40	100.0	0.0	4.108 +/- 1.076	3.340	5.580

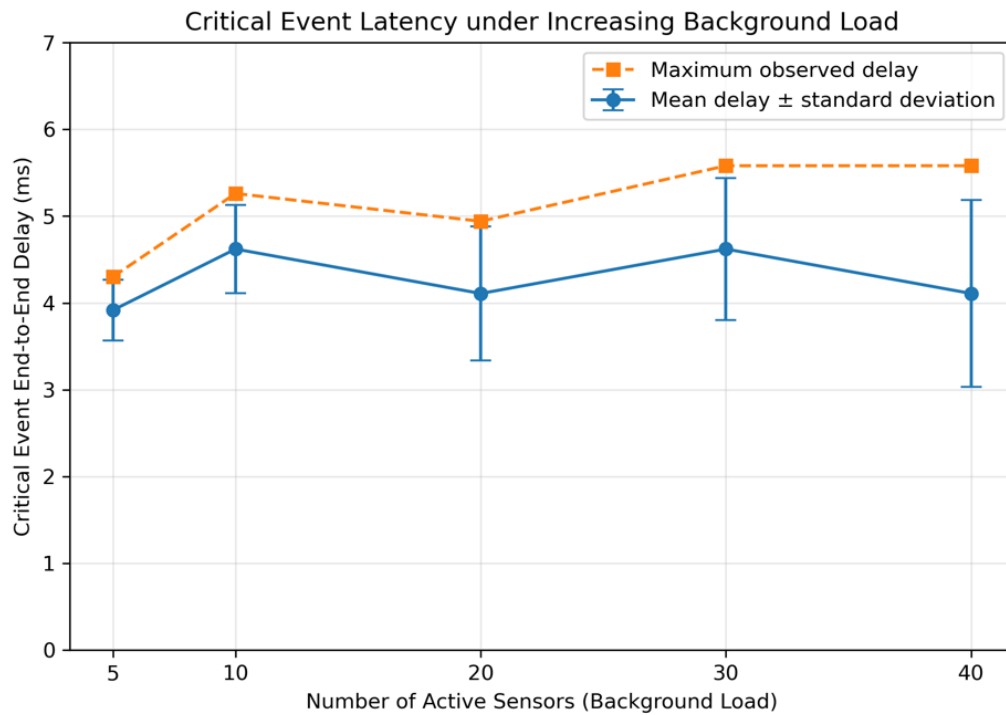


Figure 9. Critical-event end-to-end latency under increasing background load. Error bars show one standard deviation across five independent repetitions; the dashed curve shows the maximum observed delay.

6.5 Discussion

All 25 critical alarms were delivered successfully, giving a critical delivery rate of 100% and a critical loss rate of 0% at every tested load. Mean critical-event latency remained low, ranging from approximately 3.92 ms to 4.62 ms. The maximum delay observed in any completed repetition was approximately 5.58 ms.

The mean delay does not increase monotonically with N , which is expected for a small five-seed sample in a wireless CSMA/CA system with randomized sensor placement and MAC backoff. The more visible effect at higher loads is increased run-to-run variation: the standard deviation rises from about 0.35 ms at $N = 5$ to about 1.08 ms at $N = 40$. Even so, the largest observed delay remains small in absolute terms.

The adopted project design requirement is critical-event end-to-end latency < 100 ms. The worst observed value, 5.5804 ms, is only about 5.58% of that bound; therefore all 25 completed critical-event runs satisfy the requirement with a large margin. This 100 ms threshold is a project engineering target used for evaluation, not a mandatory IEEE 802.15.4 standard limit.

Experiment 3 conclusion: under the tested 50-byte/10-second background workload and $N = 5$ -40 active sensors, the urgent smoke-alarm traffic remains highly reliable and low-latency. No critical packet loss was observed, and no latency violation occurred.

6.6 Reproducibility Artifacts

Experiment 3 produced 25 scalar/vector result sets (five background-load levels $\times$ five repetitions). The critical results were exported with `opp_scavetool` and processed into `experiment3_critical_raw.csv` and `experiment3_summary.csv`; the final latency plot is `experiment3_critical_delay_final.png`. Configurations are `Critical_N5`, `Critical_N10`, `Critical_N20`, `Critical_N30`, and `Critical_N40`, each extending `CriticalEventBase` with `repeat = 5` and `seed-set = ${repetition}`.

```
opp_scavetool export -F CSV-R -o results/experiment3_critical_raw.csv ... results/Critical_N*.sca
python3 <summary/plot processing script>
```

7. Experiment 4 - Application Strategy: Request/Response vs Publish/Subscribe

7.1 Objective

This experiment compares two application communication strategies on the same IEEE 802.15.4 smart-home topology: (1) request/response polling, in which the core monitoring server actively queries each sensor and receives an echo response, and (2) a publish/subscribe-like proxy, in which sensors publish periodic updates directly to a subscriber at the monitoring server. The comparison focuses on delivery reliability, application-visible latency, application-level traffic cost, the effect of the reporting/polling interval, and behavior under the highest offered load ($T = 1$ s).

7.2 Controlled Parameters

Parameter	Setting
Active sensor nodes	12 (fixed)
Application payload	50 B (fixed)
Reporting / polling interval T	1, 5, 10, 30, 60 s
Simulation duration	600 s per run
Independent repetitions	5 per strategy and interval; seed-set = $\{\text{repetition}\}$
Request/response strategy	12 monitoring-server UdpBasicApp clients -> sensor UdpEchoApp on UDP port 2000
Publish/subscribe-like strategy	Sensor UdpBasicApp publishers -> monitoring-server UdpSink on UDP port 3000
Common topology / access	Same ApplicationStrategyNetwork, IoT gateway, PPP core link, and IEEE 802.15.4 access model

7.3 Metrics and Processing

- Request/response delivery (%) = responses received at the monitoring server / polling requests sent x 100%; loss is the complementary percentage.
- Request/response latency = rcvdPkLifetime at each monitoring-server UdpBasicApp after the packet is echoed by UdpEchoApp; this is a round-trip-time (RTT) metric.
- Publish/subscribe-like delivery (%) = publications received by monitoringServer.app[0] / publications sent by all sensors x 100%; one-way latency is endToEndDelay at the UdpSink.

- For each T, latency means are aggregated across the twelve sensor flows and then summarized across five independent repetitions; the table reports the mean and standard deviation across repetitions.
- Application-level traffic cost = generated request packets + generated echo responses for request/response, versus generated publication packets for publish/subscribe-like. This is a packet-count proxy and does not include MAC/PHY headers, retransmissions, or airtime.

7.4 Results

T (s)	RR delivery (%)	RR RTT (ms) Mean +/- SD	RR app packets / run	PUB delivery (%)	PUB delay (ms) Mean +/- SD	PUB app packets / run
1	99.978	12.461 +/- 1.237	14376	100.0	5.666 +/- 0.204	7188
5	99.958	12.458 +/- 1.279	2880	100.0	5.654 +/- 0.222	1440
10	99.944	12.429 +/- 1.260	1440	100.0	5.657 +/- 0.225	720
30	99.917	12.455 +/- 1.364	480	100.0	5.631 +/- 0.240	240
60	99.833	12.416 +/- 1.390	240	100.0	5.656 +/- 0.313	120

Table 4. Experiment 4 application-strategy results averaged over five independent repetitions.

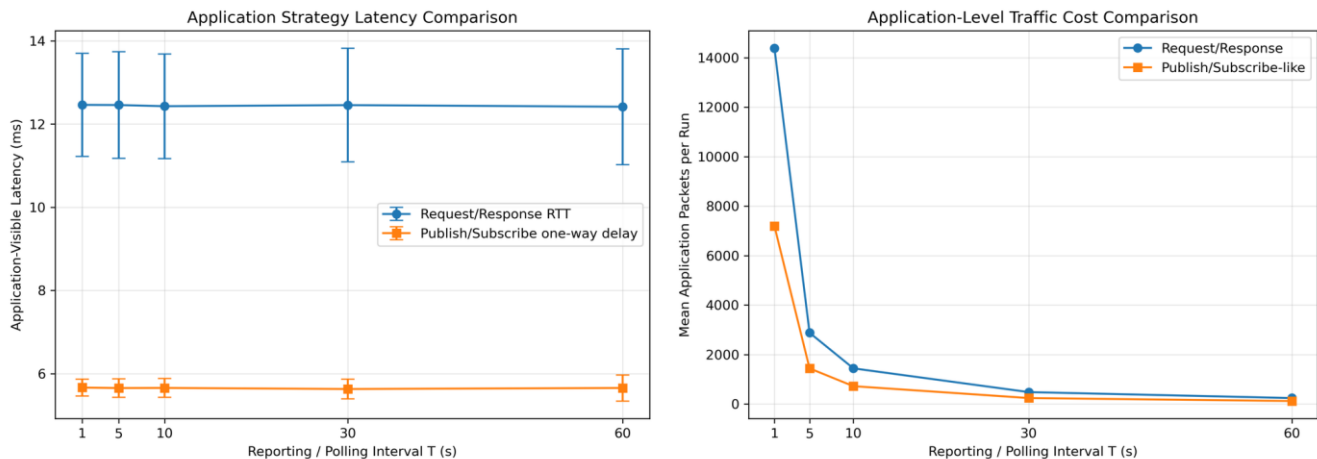


Figure 10. Application-strategy comparison: application-visible latency (left) and application-level traffic cost (right). Error bars show one standard deviation across five repetitions; request/response latency is RTT, while publish/subscribe-like latency is one-way delay.

7.5 Discussion

Publish/subscribe-like delivery was 100% for all five intervals. Request/response also remained highly reliable, ranging from 99.978% at $T = 1$ s to 99.833% at $T = 60$ s. The apparently larger percentage loss at long intervals should not be interpreted as a load-related degradation: the absolute number of missing responses was extremely small, and at $T = 60$ s a single missing response has a larger percentage effect because each run contains only 120 polling requests on average. Across all intervals, request/response delivery remained above 99.8%.

The request/response RTT was nearly independent of the polling interval, remaining between 12.416 and 12.461 ms on average. The publish/subscribe-like one-way delay was similarly stable, between 5.631 and 5.666 ms. The two latency curves should not be interpreted as a direct two-to-one speed comparison because they represent different application semantics: polling RTT contains both the request and return path, whereas publication delay contains only the publisher-to-subscriber path. Even under the highest offered load ($T = 1$ s), both strategies remained low-latency. The largest observed RR RTT was 51.172 ms, and the largest observed publication delay was 33.839 ms.

Application-level traffic cost clearly distinguishes the two strategies. For every T , request/response generated exactly twice as many application packets as the publish/subscribe-like proxy: 14,376 versus 7,188 packets at $T = 1$ s, 2,880 versus 1,440 at $T = 5$ s, and the same 2:1 ratio down to $T = 60$ s. This follows directly from the communication pattern: each poll requires a request and a response, whereas each publication requires one update packet. The result demonstrates the traffic efficiency advantage of event/data publication when continuous server polling is not required. This packet-count metric is an application-level proxy; lower-layer header bytes, retransmission cost, and radio airtime are left to the cross-layer study.

Experiment 4 conclusion: both strategies provide high reliability and stable latency over the tested interval sweep, but the publish/subscribe-like strategy reduces generated application traffic by 50% for the same update interval and achieved 100% delivery in the tested runs. Request/response remains appropriate when explicit on-demand queries and acknowledgable transaction completion are required, while publication is more traffic-efficient for regular sensor updates.

7.6 Reproducibility Artifacts

Experiment 4 produced 50 result sets (five intervals x five repetitions x two strategies). Filtered outputs are results/experiment4_rr_raw.csv and results/experiment4_pub_raw.csv; aggregate results are in results/experiment4_summary.csv. Configurations RR_T1...RR_T60 and PUB_T1...PUB_T60 are

defined in simulations/experiment4_application_strategy/omnetpp.ini. opp_scavetool CSV-R export plus the saved Python aggregation script reproduces packet counts, delivery/loss, and latency summaries.

8. Experiment 5 - Cross-Layer Packet-Size / 6LoWPAN-Inspired Study

8.1 Objective

This experiment quantifies how application payload size affects reliability, end-to-end latency, useful-bit throughput, and energy cost on the constrained IEEE 802.15.4 access network. The current IPv4/UDP INET model supplies the simulated results, while a separate RFC-based calculation estimates 6LoWPAN link fragments, link-layer transmissions, minimum serialization airtime, and the extra exposure to retransmissions. The two result types are kept explicitly separate.

8.2 Method and Metrics

Simulation setup: $N = 12$ active sensors, $T = 10$ s, 600 s per run, and payloads of 20, 50, 80, 100, 150, 250, and 500 B; each point uses five independent repetitions. PDR/loss use application packet counters, delay uses endToEndDelay statistics, throughput is delivered payload bits per run, and energy uses the same active-sensors-plus-gateway convention as Experiments 1-2.

Analytical method: RFC 6282 [10] gives about 80 B of secured IEEE 802.15.4 MAC payload, and RFC 4944 [9] requires link fragmentation when a 6LoWPAN datagram does not fit in one frame. The documented header, fragment-size, and airtime assumptions used for the calculation are given explicitly in Section 8.4.

8.3 Results

Payload (B)	PDR (%)	Delay (ms) Mean +/- SD	Max delay (ms)	Throughput (bps)	Total energy (J)	Energy/pkt (mJ)
20	100.000	4.443 +/- 0.022	5.580	192	1.191	1.654
50	100.000	5.405 +/- 0.022	6.543	480	1.658	2.302
80	100.000	6.368 +/- 0.022	7.505	768	2.124	2.950
100	100.000	10.113 +/- 0.025	12.363	960	3.152	4.378

Payload (B)	PDR (%)	Delay (ms) Mean +/- SD	Max delay (ms)	Throughput (bps)	Total energy (J)	Energy/pkt (mJ)
150	100.000	11.717 +/- 0.025	13.967	1440	3.929	5.457
250	100.000	18.019 +/- 0.032	26.767	2400	6.201	8.613
500	99.639	36.691 +/- 1.350	89.053	4782.7	12.266	17.098

Table 5. Simulated Experiment 5 packet-size results averaged over five independent repetitions.

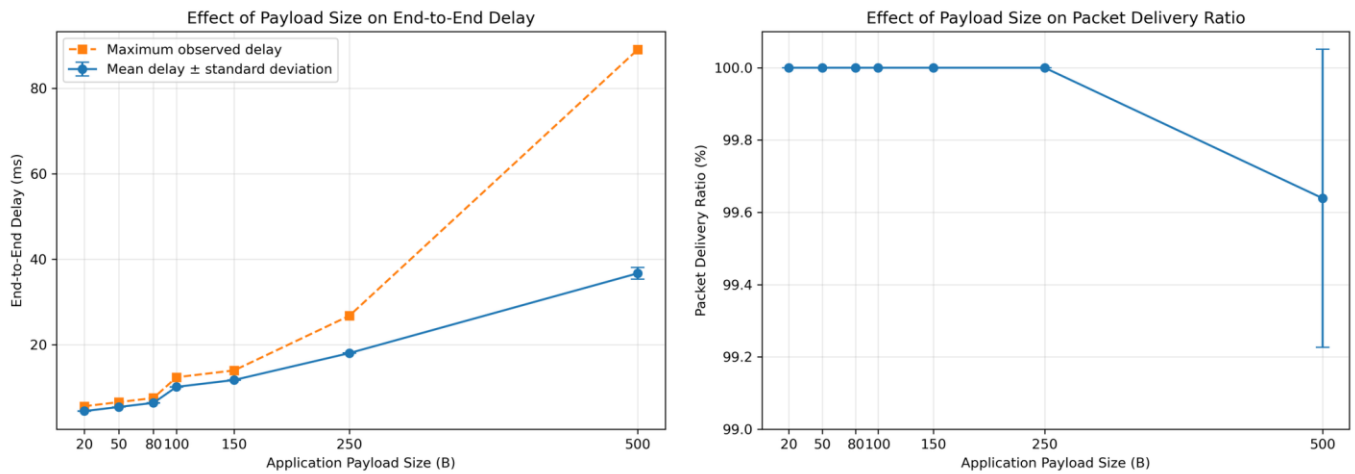


Figure 12. Payload-size effects on delay (left) and packet delivery ratio (right). Error bars show one standard deviation across five repetitions; the delay panel also shows the maximum observed packet delay.

8.4 Analytical 6LoWPAN Fragmentation and Link-Transmission Estimate

This is an analytical calculation, not a simulator trace. RFC 6282 [10] gives about 80 B of secured MAC payload. Using the documented 14 B compressed IPv6/UDP allowance, RFC 4944 [9] 4/5 B fragmentation headers, and 8-byte offset granularity, each non-final fragment carries 72 B of datagram data. Because the model has one IEEE 802.15.4 access hop, F is also the minimum number of wireless data-frame transmissions per one-way datagram before ACKs/retries.

Payload L (B)	Estimated fragments F	Min. one-way airtime (ms)	Poll+echo frames (same L)
20	1	2.59	2
50	1	3.55	2
100	2	6.94	4

200	3	11.81	6
400	6	23.20	12

Table 6. Analytical 6LoWPAN fragment/transmission estimate for the assignment payloads; values are calculations, not INET measurements.

Table 6 airtime is a 250 kbps serialization lower bound and excludes PHY preamble, CSMA/CA backoff, inter-frame spacing, ACKs, processing, and retries. The same calculation gives 2, 3, 4, and 8 fragments for simulated payloads 80, 150, 250, and 500 B. More fragments increase airtime/energy and retransmission exposure: if one frame succeeds on its first attempt with probability q , all F fragments do so with probability q^F . As an illustration only, $q = 0.99$ gives 99% for $F = 1$ but 94.1% for $F = 6$; retries improve delivery at extra airtime and energy cost.

The calculation also connects directly to polling. Experiment 4 used a 50 B request and 50 B echo; each is one frame here, so polling needs two wireless data frames per completed transaction versus one for a 50 B publication, matching the measured 2:1 packet ratio. With a small one-frame request and an F -fragment response, polling requires about $1 + F$ data frames versus F for publication before ACKs/retries. On h wireless hops the burden scales roughly with hF ; this topology has $h = 1$ before the wired PPP core.

8.5 Simulated Results vs Analytical Interpretation

Simulation and analysis are intentionally separated. In the current IPv4/UDP INET model, PDR stayed 100% through 250 B and fell to 99.639% at 500 B; mean delay rose from 4.443 to 36.691 ms (89.053 ms maximum), and energy per delivered packet rose from 1.654 to 17.098 mJ. These measured trends are qualitatively consistent with the analytical 8-fragment/airtime pressure estimated for 500 B, but they do not prove RFC 4944 FRAG1/FRAGN behavior because the 6LoWPAN adaptation layer was not simulated. Table 5 and Figure 12 are simulator-derived; Table 6, airtime bounds, and q^F examples are analytical approximations.

8.6 Reproducibility Artifacts

Experiment 5 produced 35 result sets under simulations/experiment5_packet_size/. Filtered and aggregate outputs are results/experiment5_raw.csv and results/experiment5_summary.csv; final plots are experiment5_delay.png, experiment5_pdr.png, experiment5_energy_per_packet.png, and experiment5_throughput.png. opp_scavetool plus the saved Python script reproduces the simulated results; Table 6 is reproducible from the RFC-based assumptions documented in Section 8.4.

9. References

- [1] ENCS5325 project specification, “Smart-Home IoT Network Design and Performance Evaluation Using OMNeT++,” Summer 2026.
- [2] INET Framework, IEEE 802.15.4 Smart Home Showcase:
<https://inet.omnetpp.org/docs/showcases/wireless/ieee802154/doc/>
- [3] OMNeT++ Documentation: <https://docs.omnetpp.org/>
- [4] INET Framework Documentation: <https://doc.omnetpp.org/inet/api-current/neddoc/>
- [5] INET User’s Guide, IEEE 802.15.4: <https://inet.omnetpp.org/docs/users-guide/ch-802154.html>
- [6] INET User’s Guide, Power Consumption Modeling: <https://inet.omnetpp.org/docs/users-guide/ch-power.html>
- [7] INET User’s Guide, Applications: <https://inet.omnetpp.org/docs/users-guide/ch-apps.html>
- [8] INET UdpEchoApp documentation: <https://doc.omnetpp.org/inet/api-current/neddoc/inet.applications.udpapp.UdpEchoApp.html>
- [9] IETF RFC 4944, Transmission of IPv6 Packets over IEEE 802.15.4 Networks: <https://www.rfc-editor.org/rfc/rfc4944.html>
- [10] IETF RFC 6282, Compression Format for IPv6 Datagrams over IEEE 802.15.4-Based Networks: <https://www.rfc-editor.org/rfc/rfc6282.html>